AI Recipe & Meal Planning Agent

Student: Vania Sameen

1. Project Overview

This idea is inspired from one of the examples shared in stage 1 instructions in EECS3311

1.1 Problem statement

I selected this concept because it addresses an everyday challenge that is often more complicated than it initially appears: deciding what to eat while managing time, money, dietary needs, and available ingredients.Meal planning is not always simply a matter of choosing recipes. For students, busy individuals, and households on limited budgets, it can involve balancing grocery expenses, avoiding food waste, accommodating dietary restrictions, and making the most of ingredients already available at home.

What interested me most about this project was the opportunity to design software that could make an ordinary but sometimes overwhelming responsibility more manageable. I wanted to explore how AI could offer personalized assistance without replacing the reliability and structure of conventional software.

I adapted the original project idea into a system with eleven core features, combining recipe discovery, pantry management, AI-assisted meal planning, and automatic shopping-list generation.

1.2 Target users

The intended users are students, busy home cooks and budget-conscious households, including people who want to plan around dietary preferences, available groceries and leftovers.

1.3 Proposed solution and interfaces

Users access the application through a graphical user interface (GUI) or command-line interface (CLI). It supports preference management, pantry and expiry tracking, ingredient-based recipe search, context-aware recommendations, AI recipe generation, weekly meal planning, serving adjustment, substitutions, leftover reuse, shopping lists, and natural-language meal-plan editing.

1.4 AI agent and software architecture

The agent performs a coordinated planning process rather than returning a single chatbot response. It uses recipe and pantry context, ranks candidate meals with either BudgetFirstStrategy or PantryFirstStrategy, requests proposed content through LLMClient, and checks the proposed plan through PlanValidator. If validation fails, it can request a revision; results are accepted only after successful validation. Recipe scaling, inventory changes, and shopping-list consolidation remain deterministic. The interfaces send actions to MealPlannerController, which delegates to MealPlanningFacade and the appropriate collaborators.

1.5 Proposed technology

The system will be in Java with a GUI and a command-line interface. LangChain4jLLMClient will adapt the chosen model provider to the LLMClient interface; GPT-4o mini is the proposed LLM. Recipe data can begin as a local curated collection, with an in-session profile, pantry and active meal plan. Exact GUI tooling and provider credentials will be finalized in Stage 2. Stage 1 specifies the architecture rather than claiming the system has already been built.

1.6 Scope, assumptions and limitations

I have deliberately limited the first implementation to eleven meaningful features (F01–F11) and two user interfaces, rather than implementing every feature in the sample meal-planning project.

Data and persistence: Recipes can be stored in a seeded local collection. User profile, pantry contents and the current meal plan are modeled in the running application; permanent database storage, multi-user accounts, history across sessions and synchronization across devices are outside the proposed initial scope.

External services: The only required external integration is the selected LLM through LLMClient. This design does not rely on live grocery pricing, delivery, weather or nutrition databases. Grocery cost estimates and nutrition values, if displayed, are approximate and must not be represented as verified real-time information.

Reliability and safety: An LLM can suggest incomplete, inconsistent or unsuitable recipes. PlanValidator applies checks supported by known structured data. it does not provide a medical or allergy-safety guarantee. If a provider request fails, times out or returns unusable content, the application should report the problem and leave accepted state unchanged. Unknown ingredients, units or prices should be flagged rather than invented.

Feature boundaries: F06 chooses budget-first versus pantry-first ranking; it is not a mathematical optimization guarantee. F11 supports structured replacement or removal of planned meals, not arbitrary natural-language edits. The shopping list is an estimate of missing ingredient quantities. No automatic purchases are performed.

F01 | User Preferences and Dietary Constraints

Description	Record preferences, disliked ingredients, dietary restrictions and a weekly food budget.
User interaction	GUI: Edit profile fields and save. CLI: Select profile option and enter values.
Input	Preferences; restrictions; weekly budget.
Output	Updated UserProfile and confirmation.
AI involvement	Deterministic
Expected workflow	1. Controller receives data. 2. Facade coordinates profile update. 3. UserProfile.updatePreferences() and setRestrictions() accept valid values. 4. Display confirmation.
Alternative / error cases	Reject negative budget or invalid inputs; retain existing accepted values.
Responsible UML classes	MealPlannerGUI, MealPlannerCLI, MealPlannerController, MealPlanningFacade, UserProfile

F02 | Pantry and Expiry Management

Description	Maintain pantry items with amount, unit, expiry information, type and storage notes; identify leftovers or soon-expiring items.
User interaction	GUI: Add/edit pantry items, view expiry and leftovers. CLI: Enter pantry and filtering commands.
Input	Ingredient name; quantity; unit; expiry date; kind; storage note; optional day threshold.
Output	Updated Pantry; matching PantryItem lists.
AI involvement	Deterministic
Expected workflow	1. Controller/Facade receives item. 2. PantryService.addOrUpdate(item) updates Pantry. 3. expiringWithin(days) or findLeftovers() filters items. 4. Display results.
Alternative / error cases	Reject invalid quantities or missing required fields; do not invent expiry dates.
Responsible UML classes	MealPlannerController, MealPlanningFacade, PantryService, Pantry, PantryItem

F03 | Ingredient-Based Recipe Search

Description	Find existing recipes by text terms or available pantry ingredients; does not generate new recipes.
User interaction	GUI: Enter search terms or choose pantry-based search. CLI: Enter keywords/ingredient search option.
Input	Search text or List<PantryItem>.
Output	List<Recipe> matching search criteria.
AI involvement	Deterministic
Expected workflow	1. User selects keyword or pantry-ingredient search in the GUI/CLI. 2. For keywords, MealPlannerController.searchRecipes(query) delegates to MealPlanningFacade.searchRecipes(query), then RecipeService.search(query). 3. For pantry ingredients, MealPlannerController.searchByIngredients(items) delegates to MealPlanningFacade.searchByIngredients(items), then RecipeService.findUsing(items). 4. Display the returned List<Recipe>; distinguish ingredient matches from recipes that can be cooked completely.
Alternative / error cases	For no matches, show an empty-result message; distinguish partial ingredient matches from fully cookable recipes.
Responsible UML classes	MealPlannerController, MealPlanningFacade, RecipeService, Recipe, PantryItem

F04 | Context-Aware Recipe Recommendations

Description	Rank candidate recipes using preferences, restrictions, pantry contents, budget and chosen planning priority; distinct from keyword search.
User interaction	GUI: Request personalized recommendations. CLI: Submit recommendation command.
Input	PlanRequest (including UserProfile); Pantry; recipe candidates.
Output	Ranked List<Recipe> and explanations when constraints limit options.
AI involvement	Hybrid
Expected workflow	1. Agent.recommend(req, pantry) receives context. 2. RecipeService supplies candidates. 3. MealPlanningStrategy ranks candidates. 4. CHECK the ranked recipes against known dietary restrictions and available planning constraints before displaying recommendations. 5. Present recommendations.
Alternative / error cases	If none fit, report unsatisfied constraints; do not assert allergen safety without verified information.
Responsible UML classes	MealPlanningAgent, RecipeService, MealPlanningStrategy, PlanRequest, UserProfile, Pantry, Recipe, PlanValidator, MealPlanningFacade

F05 | AI Recipe Generation

Description	Create a new recipe from a natural-language planning request and dietary context.
User interaction	GUI: Enter recipe requirements and select Generate. CLI: Submit generation request.
Input	PlanRequest; profile/restrictions; optional user instructions.
Output	Generated Recipe with ingredient requirements and preparation instructions.
AI involvement	Hybrid
Expected workflow	1. Agent.generateRecipe(req) sends context through LLMClient. 2. LangChain4jLLMClient returns content. 3. Convert the LLM generatec response into a structured Recipe object containing ingredients and preparation steps. 4. PlanValidator.checkRecipe(recipe, profile) checks known restrictions. 5. Save accepted recipe via RecipeService.saveGenerated(recipe) when requested.
Alternative / error cases	Malformed AI output, failure to contact model, conflicting restrictions: report/retry or reject; never guarantee allergy safety.
Responsible UML classes	MealPlanningAgent, LLMClient, LangChain4jLLMClient, Recipe, PlanValidator, UserProfile, RecipeService, PlanRequest

F06 | Weekly Meal-Plan Generation

Description	Produce a personalized week of planned meals using either BudgetFirstStrategy or PantryFirstStrategy.
User interaction	GUI: Select week, priority and generate plan. CLI: Enter plan request, pantry context and priority.
Input	PlanRequest (weekStart, priority, profile, instruction); Pantry; available recipes.
Output	MealPlan containing PlannedMeal entries.
AI involvement	Hybrid
Expected workflow	1. Agent.generate(req, pantry) receives inputs. 2. Use selected MealPlanningStrategy implementation to rank candidates. 3. LLMClient supports plan assembly. 4. Represent planned entries as MealPlan/PlannedMeal. 5. PlanValidator.validate(plan, req) checks constraints. 6. Revise or report failure as appropriate.
Alternative / error cases	Invalid request, insufficient recipe coverage, cost estimates or LLM failure: explain limitations; do not silently violate restrictions.
Responsible UML classes	MealPlanningAgent, MealPlanningStrategy, BudgetFirstStrategy, PantryFirstStrategy, LLMClient, PlanValidator, RecipeService, MealPlan, PlannedMeal, PlanRequest, Pantry

F07 | Recipe Serving-Size Adjustment

Description	Recalculate ingredient quantities for a selected number of portions.
User interaction	GUI: Select recipe and adjust servings. CLI: Supply recipe ID and target servings.
Input	Recipe or ID; positive target serving count.
Output	Scaled Recipe with updated ingredient amounts.
AI involvement	Deterministic
Expected workflow	1. RecipeService.getById(id), if needed, obtains the recipe. 2. RecipeService.scale(recipe, servings) computes proportional amounts. 3. Display the scaled result.
Alternative / error cases	Reject zero/negative serving counts; report unknown recipe IDs and quantities that cannot be converted reliably.
Responsible UML classes	RecipeService, Recipe, IngredientRequirement, MealPlanningFacade

F08 | Intelligent Ingredient Substitution

Description	Suggest context-aware replacement ingredients for an unavailable or unsuitable recipe ingredient.
User interaction	GUI: Select ingredient and request substitutes. CLI: Provide recipe and ingredient name.
Input	Recipe; ingredient name; available restriction context.
Output	Proposed substitute ingredients and any qualifications.
AI involvement	Hybrid
Expected workflow	1. Agent.suggestSubstitutions(recipe, ingredient) processes request. 2. LLMClient proposes alternatives. 3. Check known constraints where data is available. 4. Present qualified suggestions.
Alternative / error cases	If alternatives cannot be supported or safety data is incomplete, explain uncertainty rather than guarantee suitability.
Responsible UML classes	MealPlanningAgent, LLMClient, Recipe, UserProfile, PlanValidator, MealPlanningFacade

F09 | Leftover-Based Meal Planning

Description	Identify pantry leftovers and recommend a recipe that can use them.
User interaction	GUI: Select Use Leftovers. CLI: Run leftovers-planning option.
Input	List<PantryItem> with leftover status.
Output	Proposed Recipe incorporating suitable leftovers.
AI involvement	Hybrid

Expected workflow	1. PantryService.findLeftovers() retrieves items. 2. Facade coordinates inputs. 3. The Facade supplies a PlanRequest containing the relevant user preferences and constraints. Agent.planFromLeftovers(items,req) proposes a Recipe. 4. Display recipe and ingredients needed.
Alternative / error cases	No leftovers: inform user; expired/unsafe ingredients are not presumed edible; model failures are reported.
Responsible UML classes	PantryService, PantryItem, MealPlanningFacade, MealPlanningAgent, Recipe

F10 | Automatic Shopping-List Generation

Description	Calculate ingredients needed for a meal plan after considering pantry quantities; merge repeated entries.
User interaction	GUI: Generate/view the list for a plan. CLI: Request shopping list for active plan.
Input	MealPlan; Pantry; Recipe ingredient requirements.
Output	ShoppingList containing consolidated ShoppingListItem entries.
AI involvement	Deterministic
Expected workflow	1. ShoppingListService.build(plan, pantry) collects requirements. 2. missingFor(recipe, pantry) determines deficits. 3. consolidate(items) merges duplicates. 4. Show ShoppingList. 5. Following meal-plan changes, MealPlanObserver notification triggers ShoppingListSynchronizer to recalculate.
Alternative / error cases	Empty plan: empty list or message; unknown units/quantities: flag instead of silently computing invalid totals.
Responsible UML classes	ShoppingListService, ShoppingListSynchronizer, MealPlan, MealPlanObserver, Recipe, IngredientRequirement, Pantry, ShoppingList, ShoppingListItem

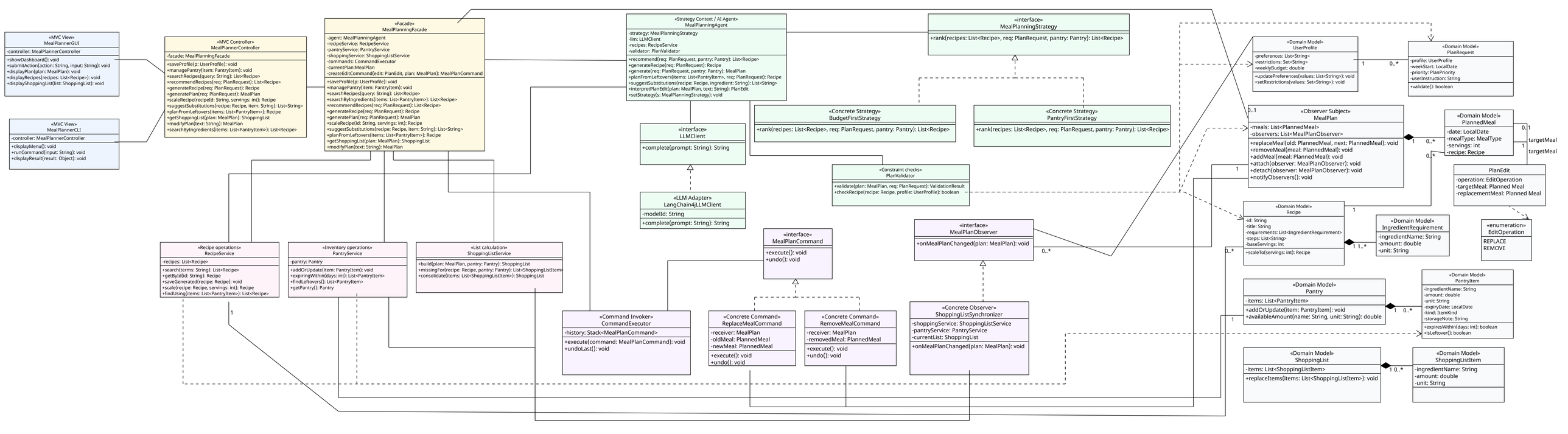
F11 | Natural-Language Meal-Plan Modification

Description	Interpret a user instruction to REPLACE or REMOVE a scheduled meal, then execute an undoable deterministic edit.
User interaction	GUI: Enter instruction within active plan. CLI: Enter edit instruction for active plan.
Input	Active MealPlan; instruction text; identified target PlannedMeal; optional replacement PlannedMeal.
Output	Updated MealPlan and synchronized shopping list; undoable edit in command history.
AI involvement	Hybrid
Expected workflow	1. Controller.modifyPlan(text) calls Facade.modifyPlan(text). 2. Facade reads its currentPlan: MealPlan; if absent, report that no active plan exists. 3. Agent.interpretPlanEdit(currentPlan, text) returns PlanEdit, specifying REPLACE or REMOVE, targetMeal, and optional replacementMeal. 4. Facade verifies the edit and createEditCommand(edit, currentPlan) produces MealPlanCommand. 5. CommandExecutor.execute(command) invokes ReplaceMealCommand or RemoveMealCommand. 6. The modified MealPlan notifies registered observers; ShoppingListSynchronizer refreshes the list. 7. Return the updated currentPlan. generatePlan(req) must assign its successful result to currentPlan.
Alternative / error cases	No current plan: request plan generation. Ambiguous target, unsupported edit, invalid replacement, or LLM failure: reject or clarify without modifying the plan. Undo is supported through CommandExecutor.undoLast(). Replacement validation and observer registration must be completed in the implementation workflow.
Responsible UML classes	MealPlannerController, MealPlanningFacade, MealPlanningAgent, LLMClient, PlanEdit, EditOperation, CommandExecutor, MealPlanCommand, ReplaceMealCommand, RemoveMealCommand, MealPlan, PlannedMeal, MealPlanObserver, ShoppingListSynchronizer, ShoppingListService, PantryService

3. UML Class Diagram

3.1 Some notes for diagrams

Class diagram is a pdf and may require zooming in due to the size of it. Some diagrams are in landscape to accommodate the layout


«MVC View»
MealPlannerGUI
-controller: MealPlannerController
+showDashboard(): void
+submitAction(action: String, input: String): void
+displayPlan(plan: MealPlan): void
+displayRecipes(recipes: List<Recipe>): void
+displayShoppingList(list: ShoppingList): void
«MVC View»
MealPlannerCLI
-controller: MealPlannerController
+displayMenu(): void
+runCommand(input: String): void
+displayResult(result: Object): void
«MVC Controller»
MealPlannerController
-facade: MealPlanningFacade
+saveProfile(p: UserProfile): void
+manageePantry(item: PantryItem): void
+searchRecipes(query: String): List<Recipe>
+recommendRecipes(req: PlanRequest): List<Recipe>
+generateRecipe(req: PlanRequest): Recipe
+generatePlan(req: PlanRequest): MealPlan
+scaleRecipe(recipeId: String, servings: int): Recipe
+suggestSubstitutions(recipe: Recipe, item: String): List<String>
+planFromLeftovers(items: List<PantryItem>): Recipe
+getShoppingList(plan: MealPlan): ShoppingList
+modifyPlan(text: String): MealPlan
+searchByIngredients(items: List<PantryItem>): List<Recipe>
«Facade»
MealPlanningFacade
-agent: MealPlanningAgent
-recipeService: RecipeService
-pantryService: PantryService
-shoppingService: ShoppingListService
-commands: CommandExecutor
-currentPlan:MealPlan
-createEditCommand(edit: PlanEdit, plan: MealPlan): MealPlanCommand
+saveProfile(p: UserProfile): void
+manageePantry(item: PantryItem): void
+searchRecipes(query: String): List<Recipe>
+searchByIngredients(items: List<PantryItem>): List<Recipe>
+recommendRecipes(req: PlanRequest): List<Recipe>
+generateRecipe(req: PlanRequest): Recipe
+generatePlan(req: PlanRequest): MealPlan
+scaleRecipe(id: String, servings: int): Recipe
+suggestSubstitutions(recipe: Recipe, item: String): List<String>
+planFromLeftovers(items: List<PantryItem>): Recipe
+getShoppingList(plan: MealPlan): ShoppingList
+modifyPlan(text: String): MealPlan
«Strategy Context / AI Agent»
MealPlanningAgent
-strategy: MealPlanningStrategy
-llm: LLMClient
-recipes: RecipeService
-validator: PlanValidator
+recommend(req: PlanRequest, pantry: Pantry): List<Recipe>
+generateRecipe(req: PlanRequest): Recipe
+generate(req: PlanRequest, pantry: Pantry): MealPlan
+planFromLeftovers(items: List<PantryItem>, req: PlanRequest): Recipe
+suggestSubstitutions(recipe: Recipe, ingredient: String): List<String>
+interpretPlanEdit(plan: MealPlan, text: String): PlanEdit
+setStrategy(s: MealPlanningStrategy): void
«interface»
MealPlanningStrategy
+rank(recipes: List<Recipe>, req: PlanRequest, pantry: Pantry): List<Recipe>
«Concrete Strategy»
BudgetFirstStrategy
+rank(recipes: List<Recipe>, req: PlanRequest, pantry: Pantry): List<Recipe>
«Concrete Strategy»
PantryFirstStrategy
+rank(recipes: List<Recipe>, req: PlanRequest, pantry: Pantry): List<Recipe>
«interface»
LLMClient
+complete(prompt: String): String
«LLM Adapter»
LangChain4jLLMClient
-modelId: String
+complete(prompt: String): String
«Constraint checks»
PlanValidator
+validate(plan: MealPlan, req: PlanRequest): ValidationResult
+checkRecipe(recipe: Recipe, profile: UserProfile): boolean
«Domain Model»
UserProfile
-preferences: List<String>
-restrictions: Set<String>
-weeklyBudget: double
+updatePreferences(values: List<String>): void
+setRestrictions(values: Set<String>): void
«Domain Model»
PlanRequest
-profile: UserProfile
-weekStart: LocalDate
-priority: PlanPriority
-userInstruction: String
+validate(): boolean
«Observer Subject»
MealPlan
-meals: List<PlannedMeal>
-observers: List<MealPlanObserver>
+replaceMeal(old: PlannedMeal, next: PlannedMeal): void
+removeMeal(meal: PlannedMeal): void
+addMeal(meal: PlannedMeal): void
+attach(observer: MealPlanObserver): void
+detach(observer: MealPlanObserver): void
+notifyObservers(): void
«Domain Model»
PlannedMeal
-date: LocalDate
-mealType: MealType
-servings: int
-recipe: Recipe
targetMeal
targetMeal
PlanEdit
-operation: EditOperation
-targetMeal: Planned Meal
-replacementMeal: Planned Meal
«enumeration»
EditOperation
REPLACE
REMOVE
«Domain Model»
Recipe
-id: String
-title: String
-requirements: List<IngredientRequirement>
-steps: List<String>
-baseServings: int
+scaleTo(servings: int): Recipe
«Domain Model»
IngredientRequirement
-ingredientName: String
-amount: double
-unit: String
«Domain Model»
Pantry
-items: List<PantryItem>
+addOrUpdate(item: PantryItem): void
+availableAmount(name: String, unit: String): double
«Domain Model»
PantryItem
-ingredientName: String
-amount: double
-unit: String
-expiryDate: LocalDate
-kind: ItemKind
-storageNote: String
+expiresWithin(days: int): boolean
+isLeftover(): boolean
«Domain Model»
ShoppingList
-items: List<ShoppingListItem>
+replaceItems(items: List<ShoppingListItem>): void
«Domain Model»
ShoppingListItem
-ingredientName: String
-amount: double
-unit: String
«Recipe operations»
RecipeService
-recipes: List<Recipe>
+search(terms: String): List<Recipe>
+getById(id: String): Recipe
+saveGenerated(recipe: Recipe): void
+scale(recipe: Recipe, servings: int): Recipe
+findUsing(items: List<PantryItem>): List<Recipe>
«Inventory operations»
PantryService
-pantry: Pantry
+addOrUpdate(item: PantryItem): void
+expiringWithin(days: int): List<PantryItem>
+findLeftovers(): List<PantryItem>
+getPantry(): Pantry
«List calculation»
ShoppingListService
+build(plan: MealPlan, pantry: Pantry): ShoppingList
+missingFor(recipe: Recipe, pantry: Pantry): List<ShoppingListItem>
+consolidate(items: List<ShoppingListItem>): ShoppingList
«interface»
MealPlanCommand
+execute(): void
+undo(): void
«Command Invoker»
CommandExecutor
-history: Stack<MealPlanCommand>
+execute(command: MealPlanCommand): void
+undoLast(): void
«Concrete Command»
ReplaceMealCommand
-receiver: MealPlan
-oldMeal: PlannedMeal
-newMeal: PlannedMeal
+execute(): void
+undo(): void
«Concrete Command»
RemoveMealCommand
-receiver: MealPlan
-removedMeal: PlannedMeal
+execute(): void
+undo(): void
«interface»
MealPlanObserver
+onMealPlanChanged(plan: MealPlan): void
«Concrete Observer»
ShoppingListSynchronizer
-shoppingService: ShoppingListService
-pantryService: PantryService
-currentList: ShoppingList
+onMealPlanChanged(plan: MealPlan): void

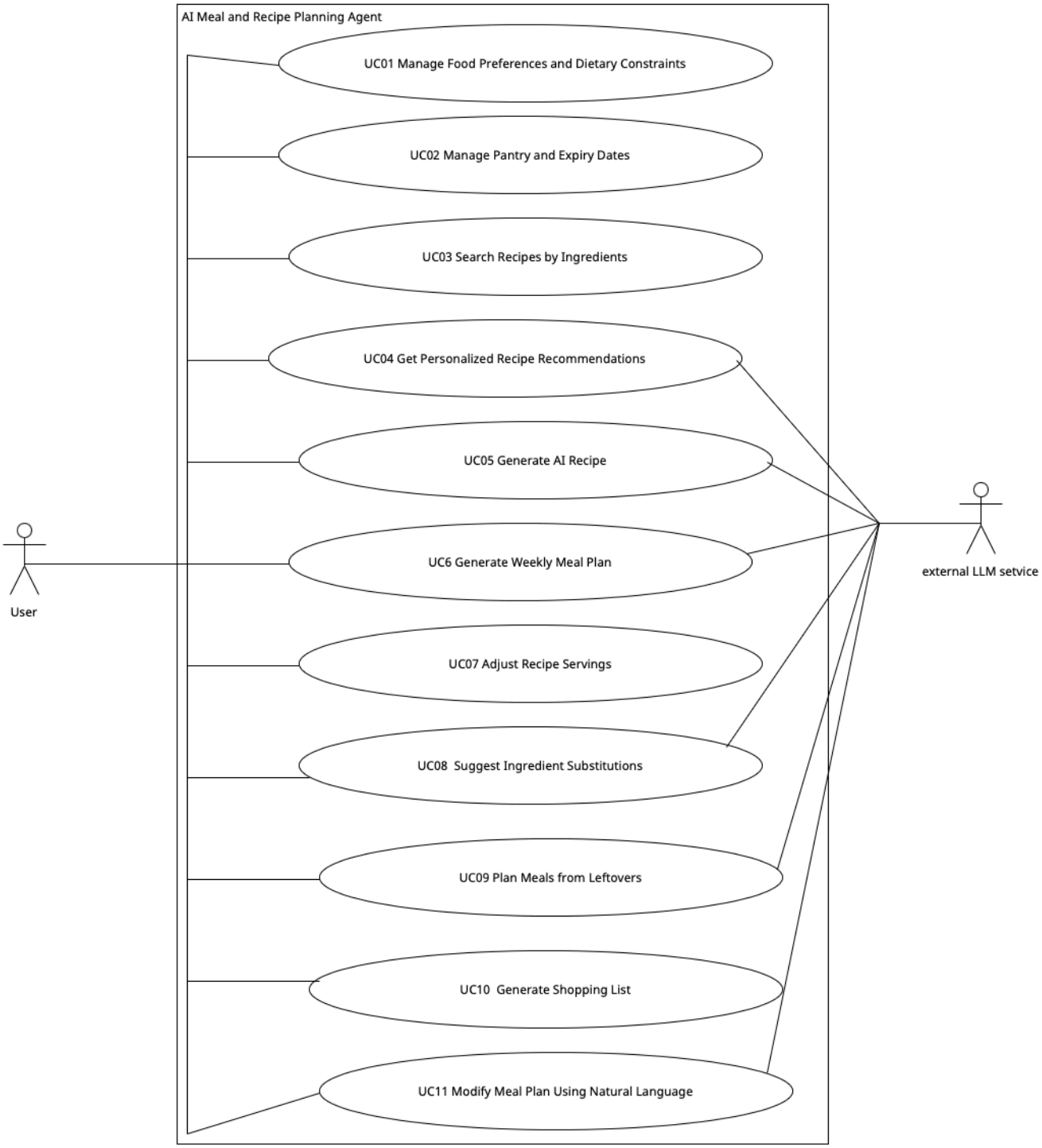
The diagram is organized into Presentation/UI, Application Control, Services, AI & Planning, Command & Observer, and Domain Models. Its background colours distinguish architectural groups, not design patterns.

The five design patterns are identified through participating classes and their connectors, then explained in Section 7.

To keep the full class diagram readable, it intentionally omits some dashed dependencies that are already clear from method parameters and return types .

PlanEdit and EditOperation represent the structured result of interpreting a natural-language meal-plan change. Their role is to connect AI interpretation to the existing Command implementation.

4. Use-Case Diagram



All eleven user goals are independently accessible. no «include» or «extend» arrows are used because the modeled workflows do not establish mandatory reusable use cases or conditional extensions of complete use cases.

5. Detailed Use-Case Descriptions — UC01–UC11

Primary actor: User. Supporting actor: External LLM Service for UC04, UC05, UC06, UC08, UC09 and UC11.

UC01 Manage Food Preferences and Dietary Constraints

Use Case ID	UC01
Use Case Name	Manage Food Preferences and Dietary Constraints
Related Feature	F01
Actor(s)	User
Goal	Update the user profile with preferences, restrictions and a weekly budget.
Preconditions	Application is running; a UserProfile is available.
Trigger	User chooses to edit preferences.
Main Success Scenario	User opens preferences; enters values; submits; application validates entries; UserProfile.updatePreferences() and setRestrictions() update valid values; interface confirms the new profile.
Alternative / Exception Flows	Invalid budget or contradictory restriction: explain and reject affected changes without corrupting accepted values.
Postconditions	UserProfile holds the accepted updates for the current application state.
Class / Method Trace	MealPlannerGUI / MealPlannerCLI; MealPlannerController.saveProfile(); MealPlanningFacade.saveProfile(); UserProfile.updatePreferences(), setRestrictions().

UC02 Manage Pantry and Expiry Dates

Use Case ID	UC02
Use Case Name	Manage Pantry and Expiry Dates
Related Feature	F02
Actor(s)	User
Goal	Maintain inventory quantities and identify ingredients nearing expiry or marked as leftovers.
Preconditions	Application is running; a Pantry is available.
Trigger	User adds/updates an ingredient or requests a pantry overview.
Main Success Scenario	User supplies PantryItem information; system checks required fields and amounts; controller delegates management; PantryService.addOrUpdate() updates Pantry; for relevant views, PantryService.expiringWithin() or findLeftovers() returns matching items; interface displays results.
Alternative / Exception Flows	Negative amounts, unsupported units, or missing information: reject or request correction; unknown expiry must not be fabricated.
Postconditions	Pantry reflects accepted inventory changes; requested inventory results are displayed.
Class / Method Trace	MealPlannerController.managePantry(); MealPlanningFacade.managePantry(); PantryService.addOrUpdate(), expiringWithin(), findLeftovers(); Pantry.addOrUpdate().

UC03 Search Recipes by Ingredients

Use Case ID	UC03
Use Case Name	Search Recipes by Ingredients
Related Feature	F03
Actor(s)	User
Goal	Find existing recipes using keyword search or available pantry ingredients.
Preconditions	RecipeService has an available recipe collection; application is running.
Trigger	User submits a recipe keyword or ingredient-based request.

Main Success Scenario	1. User selects keyword or ingredient-based recipe search. 2. For keywords: Controller.searchRecipes(query) → Facade.searchRecipes(query) → RecipeService.search(query). 3. For ingredients: Controller.searchByIngredients(items) → Facade.searchByIngredients(items) → RecipeService.findUsing(items). 4. The matching List<Recipe> is returned to the interface and displayed.
Alternative / Exception Flows	No matches: show empty results and allow revised search; ingredient-based search does not guarantee that all recipe quantities are available.
Postconditions	Matching available recipes, or a clear no-results message, are displayed.
Class / Method Trace	MealPlannerController.searchRecipes(), searchByIngredients(); MealPlanningFacade.searchRecipes(), searchByIngredients(); RecipeService.search(), findUsing(); Recipe; PantryItem.

UC04 Get Personalized Recipe Recommendations

Use Case ID	UC04
Use Case Name	Get Personalized Recipe Recommendations
Related Feature	F04
Actor(s)	User; External LLM Service
Goal	Recommend recipes reflecting user constraints, pantry context, and planning priorities.
Preconditions	A usable PlanRequest/profile and relevant recipe/pantry context exist.
Trigger	User requests personalized recommendations.
Main Success Scenario	User submits preferences and planning context; controller forwards request to facade; MealPlanningAgent.recommend(req, pantry) obtains recipe candidates; the active MealPlanningStrategy.rank() prioritizes alternatives; AI-assisted reasoning may use the external LLM; constraints are checked against available information; ranked recipes are displayed.
Alternative / Exception Flows	No acceptable candidates or unavailable LLM: explain limitation; do not label unverified recipes allergen-safe.
Postconditions	Ranked recommendations are displayed or a constraint-related error is reported.
Class / Method Trace	MealPlannerController.recommendRecipes(); MealPlanningFacade.recommendRecipes(); MealPlanningAgent.recommend(); MealPlanningStrategy.rank(); RecipeService; PlanValidator.

UC05 Generate AI Recipe

Use Case ID	UC05
Use Case Name	Generate AI Recipe
Related Feature	F05
Actor(s)	User; External LLM Service
Goal	Generate a recipe from a request while checking available restrictions.
Preconditions	A valid planning request exists; LLM service is available for generation.
Trigger	User asks to generate a new recipe.
Main Success Scenario	User supplies requirements; controller and facade call MealPlanningAgent.generateRecipe(req); agent sends context through LLMClient.complete(); output is structured into Recipe; PlanValidator.checkRecipe() checks supported dietary information; accepted recipe is returned and can be retained using RecipeService.saveGenerated().
Alternative / Exception Flows	Malformed LLM response, API failure, or known restriction violation: reject, report, or revise the proposal; do not guarantee food safety.
Postconditions	A valid candidate Recipe is presented, or the generation failure is explained.
Class / Method Trace	MealPlannerController.generateRecipe(); MealPlanningFacade.generateRecipe(); MealPlanningAgent.generateRecipe(); LLMClient.complete(); PlanValidator.checkRecipe(); RecipeService.saveGenerated().

UC06 Generate Weekly Meal Plan

Use Case ID	UC06
Use Case Name	Generate Weekly Meal Plan
Related Feature	F06
Actor(s)	User; External LLM Service

Goal	Produce a weekly MealPlan respecting preferences, available ingredients and selected priority.
Preconditions	A valid PlanRequest and relevant pantry/profile context are available.
Trigger	User selects Generate Weekly Meal Plan.
Main Success Scenario	User specifies week, restrictions and priority; controller and facade delegate to MealPlanningAgent.generate(req, pantry); agent uses BudgetFirstStrategy or PantryFirstStrategy via MealPlanningStrategy.rank(); LLMClient contributes a proposed plan; proposal is represented as MealPlan and PlannedMeal; PlanValidator.validate(plan, req) is used; approved result is displayed.
Alternative / Exception Flows	Unachievable constraints or failed validation: attempt a revised proposal or explain inability; invalid request and LLM failure receive feedback.
Postconditions	Generated and checked weekly MealPlan is displayed, or no valid plan is reported.
Class / Method Trace	MealPlannerController.generatePlan(); MealPlanningFacade.generatePlan(); MealPlanningAgent.generate(); MealPlanningStrategy.rank(); PlanValidator.validate(); MealPlan; PlannedMeal.

UC07 Adjust Recipe Servings

Use Case ID	UC07
Use Case Name	Adjust Recipe Servings
Related Feature	F07
Actor(s)	User
Goal	Recalculate the ingredient amounts for a selected recipe and target serving count.
Preconditions	A Recipe with a positive base serving count exists.
Trigger	User enters desired servings for a recipe.
Main Success Scenario	User selects recipe and serving quantity; controller forwards scale request; facade delegates to RecipeService.scale(recipe, servings), using Recipe.scaleTo(servings); IngredientRequirement quantities are scaled; adjusted recipe is displayed.
Alternative / Exception Flows	Unknown recipe, nonpositive serving count, or invalid source amounts: show validation error and preserve prior values.
Postconditions	Scaled recipe is displayed; original recipe need not be overwritten.
Class / Method Trace	MealPlannerController.scaleRecipe(); MealPlanningFacade.scaleRecipe(); RecipeService.scale(), getById(); Recipe.scaleTo().

UC08 Suggest Ingredient Substitutions

Use Case ID	UC08
Use Case Name	Suggest Ingredient Substitutions
Related Feature	F08
Actor(s)	User; External LLM Service
Goal	Propose ingredient alternatives for a selected recipe ingredient.
Preconditions	A Recipe and target ingredient are identified; relevant restriction context is accessible.
Trigger	User requests a replacement for one ingredient.
Main Success Scenario	User identifies ingredient; controller/facade call MealPlanningAgent.suggestSubstitutions(recipe, ingredient); agent uses LLMClient for candidate alternatives; results are checked against available known restrictions; suggestions are presented without automatically changing the recipe.
Alternative / Exception Flows	No reasonable substitute, unavailable model, or insufficient dietary detail: report uncertainty and avoid claiming guaranteed allergen safety.
Postconditions	List of suggested substitutions or an explanatory failure.
Class / Method Trace	MealPlannerController.suggestSubstitutions(); MealPlanningFacade.suggestSubstitutions(); MealPlanningAgent.suggestSubstitutions(); LLMClient.complete(). Access to profile during validation remains a documented assumption.

UC09 Plan Meals from Leftovers

Use Case ID	UC09
Use Case Name	Plan Meals from Leftovers
Related Feature	F09
Actor(s)	User; External LLM Service
Goal	Propose a recipe using leftover pantry ingredients.

Preconditions	Pantry exists and leftovers can be identified.
Trigger	User requests a meal using leftovers.
Main Success Scenario	User requests leftover planning; PantryService.findLeftovers() returns PantryItem entries marked as leftovers; facade passes the items to MealPlanningAgent.planFromLeftovers(items, req). The Facade prepares the PlanRequest using the available user profile and planning context before invoking the Agent. AI-assisted selection or generation returns a Recipe; user reviews the proposal.
Alternative / Exception Flows	No leftovers, questionable freshness or no compatible recipe: report limitation; expired items must not be automatically recommended.
Postconditions	A proposed Recipe using identified leftovers is shown, or the request fails safely.
Class / Method Trace	PantryService.findLeftovers(); MealPlannerController.planFromLeftovers(); MealPlanningFacade.planFromLeftovers(); MealPlanningAgent.planFromLeftovers().

UC10 Generate Shopping List

Use Case ID	UC10
Use Case Name	Generate Shopping List
Related Feature	F10
Actor(s)	User
Goal	Calculate missing ingredients for planned meals, consolidate duplicates and reflect later plan changes.
Preconditions	A MealPlan and Pantry are available.
Trigger	User requests a shopping list for an existing plan.
Main Success Scenario	User selects shopping-list generation; facade coordinates ShoppingListService.build(plan, pantry); missingFor(recipe, pantry) estimates missing items; consolidate(items) merges duplicate ingredient entries; ShoppingList is shown; later MealPlan changes notify observers and ShoppingListSynchronizer recalculates its current list.
Alternative / Exception Flows	Empty plan or complete pantry: list may be empty; incompatible units or incomplete amounts must be reported rather than silently guessed.
Postconditions	ShoppingList of needed ingredients is displayed and can be synchronized after supported meal-plan edits.
Class / Method Trace	MealPlanningFacade.getShoppingList(); ShoppingListService.build(), missingFor(), consolidate(); MealPlan.notifyObservers(); ShoppingListSynchronizer.onMealPlanChanged().

UC11 Modify Meal Plan Using Natural Language

Use Case ID	UC11
Use Case Name	Modify Meal Plan Using Natural Language
Related Feature	F11
Actor(s)	User; External LLM Service
Goal	Apply a supported natural-language meal replacement/removal through Commands, preserving undo history.
Preconditions	The Facade's currentPlan references an existing MealPlan; the user has entered an editing instruction.
Trigger	User enters a natural-language instruction to change a meal.
Main Success Scenario	1. User submits text; Controller.modifyPlan(text) calls Facade.modifyPlan(text). 2. Facade reads currentPlan and invokes Agent.interpretPlanEdit(currentPlan, text). 3. Agent returns a structured PlanEdit identifying an EditOperation, targetMeal, and optional replacementMeal. 4. Facade checks the edit and calls createEditCommand(edit, currentPlan), yielding ReplaceMealCommand or RemoveMealCommand. 5. CommandExecutor.execute(command) modifies currentPlan. 6. MealPlan.notifyObservers() informs attached MealPlanObserver instances; ShoppingListSynchronizer recalculates the shopping list. 7. Facade returns currentPlan.
Alternative / Exception Flows	No currentPlan: tell the user to generate a meal plan. Ambiguous/unsupported edits, invalid replacements, and model errors: explain or request clarification without modifying the plan. A successfully executed edit can be undone through CommandExecutor.undoLast().
Postconditions	Supported edit is executed and shopping list refreshed; rejected edits leave plan unchanged.
Class / Method Trace	MealPlannerController.modifyPlan(); MealPlanningFacade.currentPlan, modifyPlan(), createEditCommand(), generatePlan(); MealPlanningAgent.interpretPlanEdit(); PlanEdit; EditOperation;

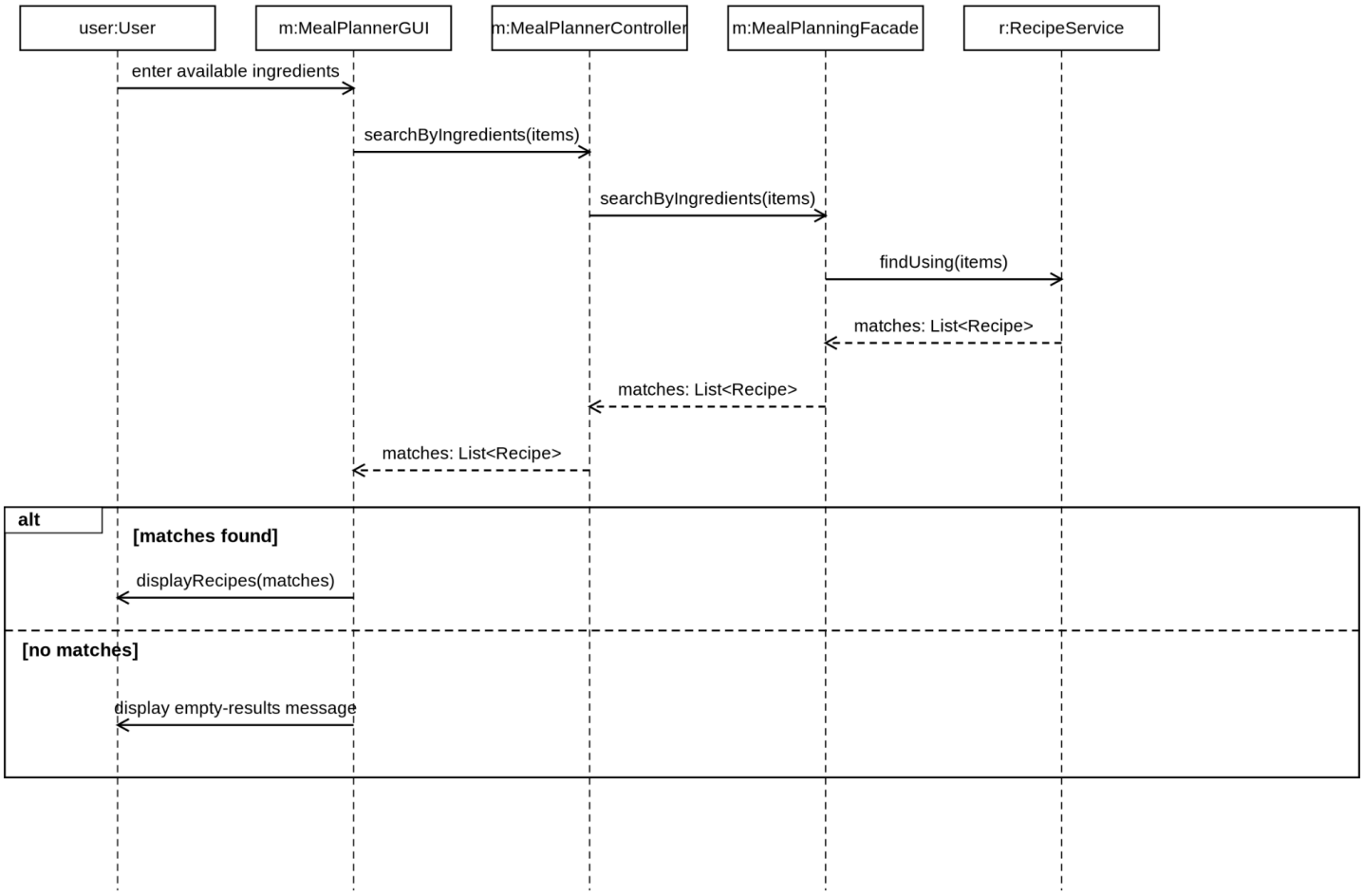
	CommandExecutor.execute(), undoLast(); ReplaceMealCommand / RemoveMealCommand; MealPlan.notifyObservers(), attach(); ShoppingListSynchronizer.onMealPlanChanged().

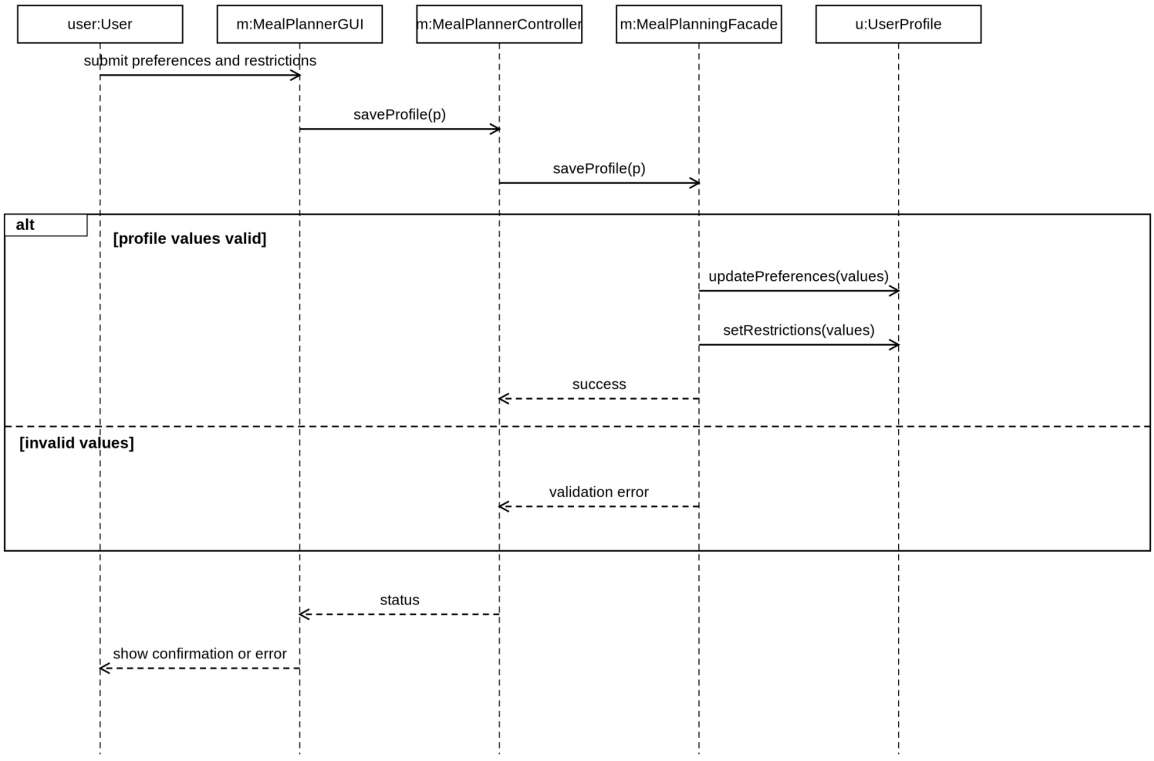
6. UML Sequence Diagrams

SD01 UC06 Weekly plan generation: Strategy, LLM proposal, validation and revision



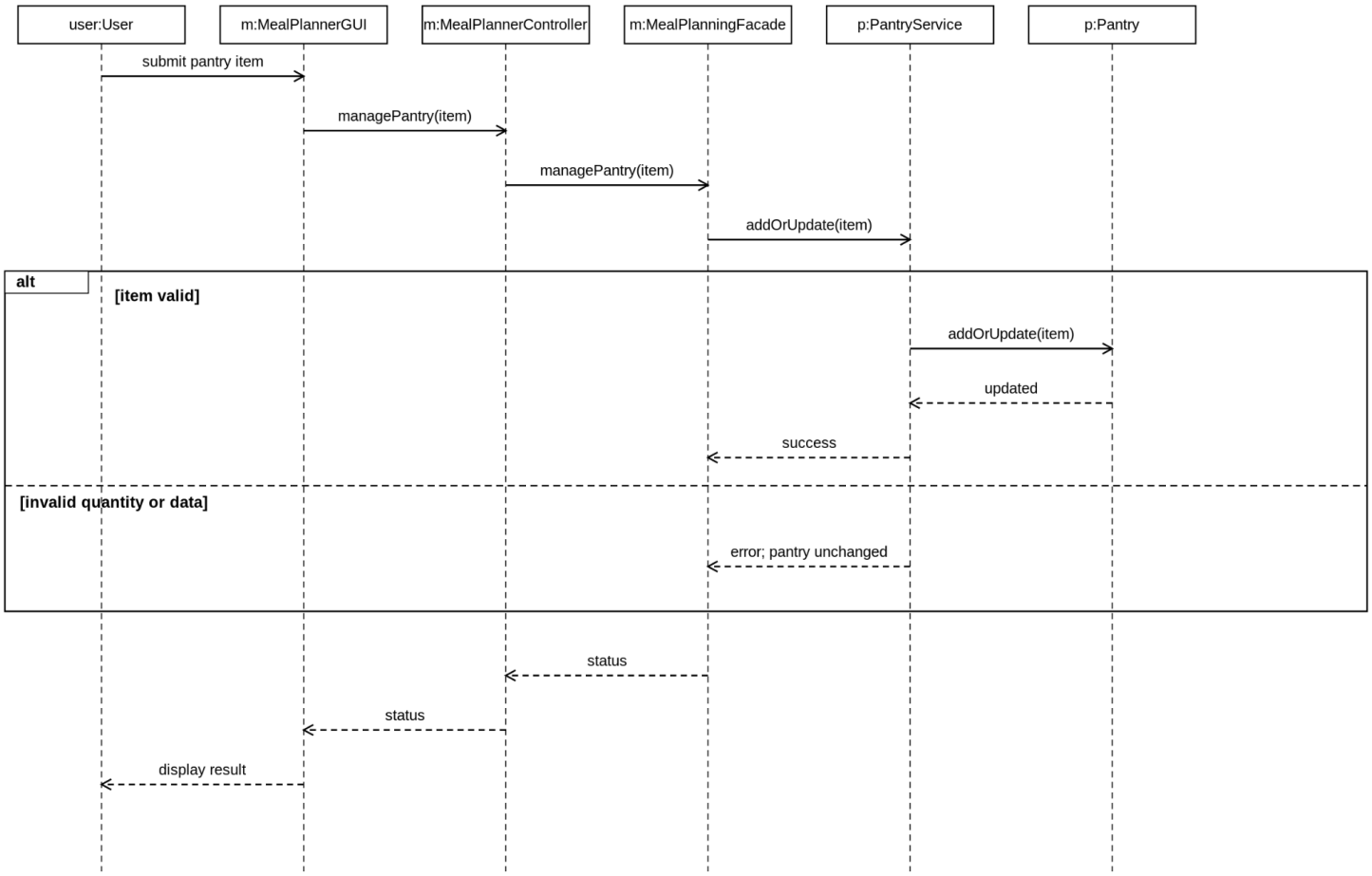
SD02 UC01–UC03 Preferences/Pantry and recipe search



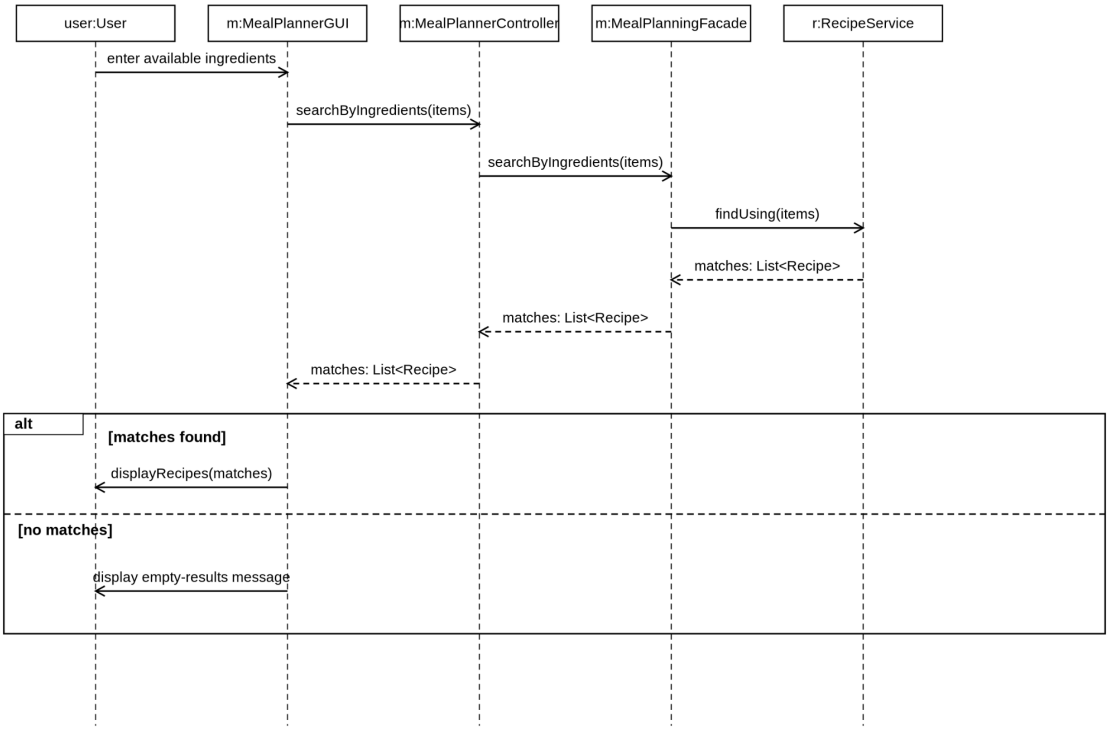


SD02-A UC01

SD02-B UC02

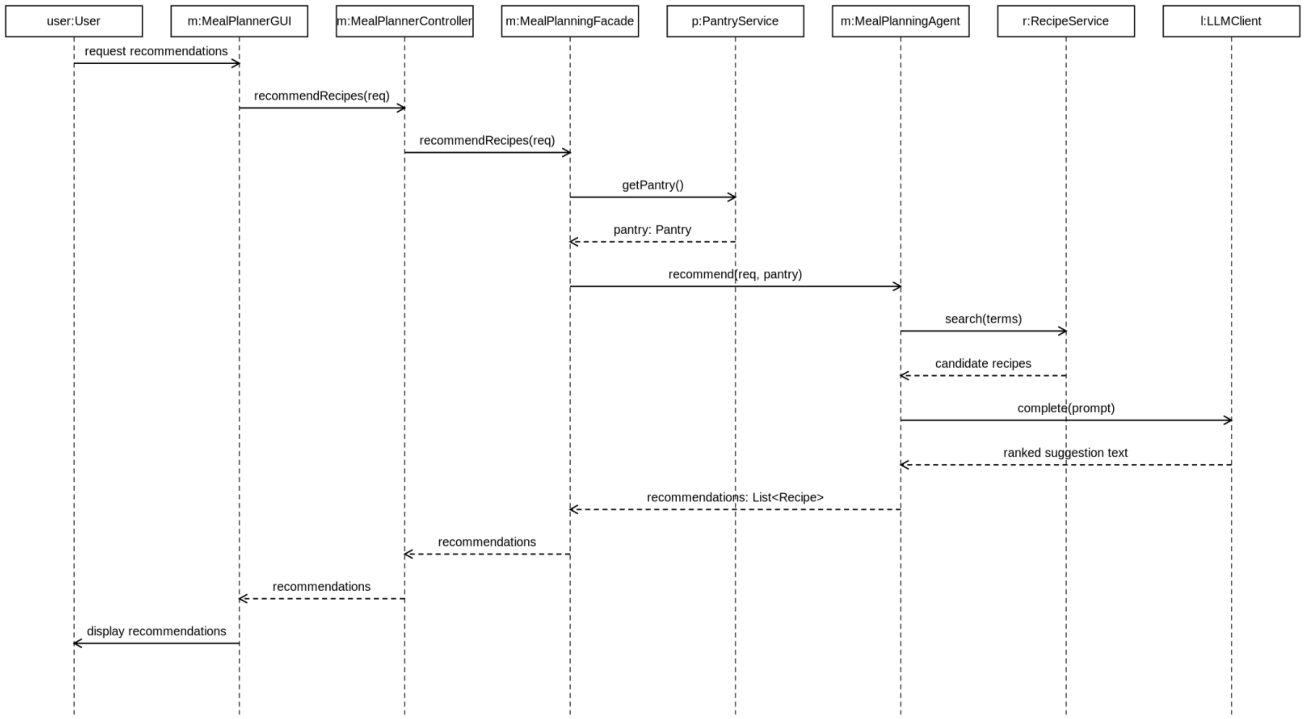


SD02-C UC03

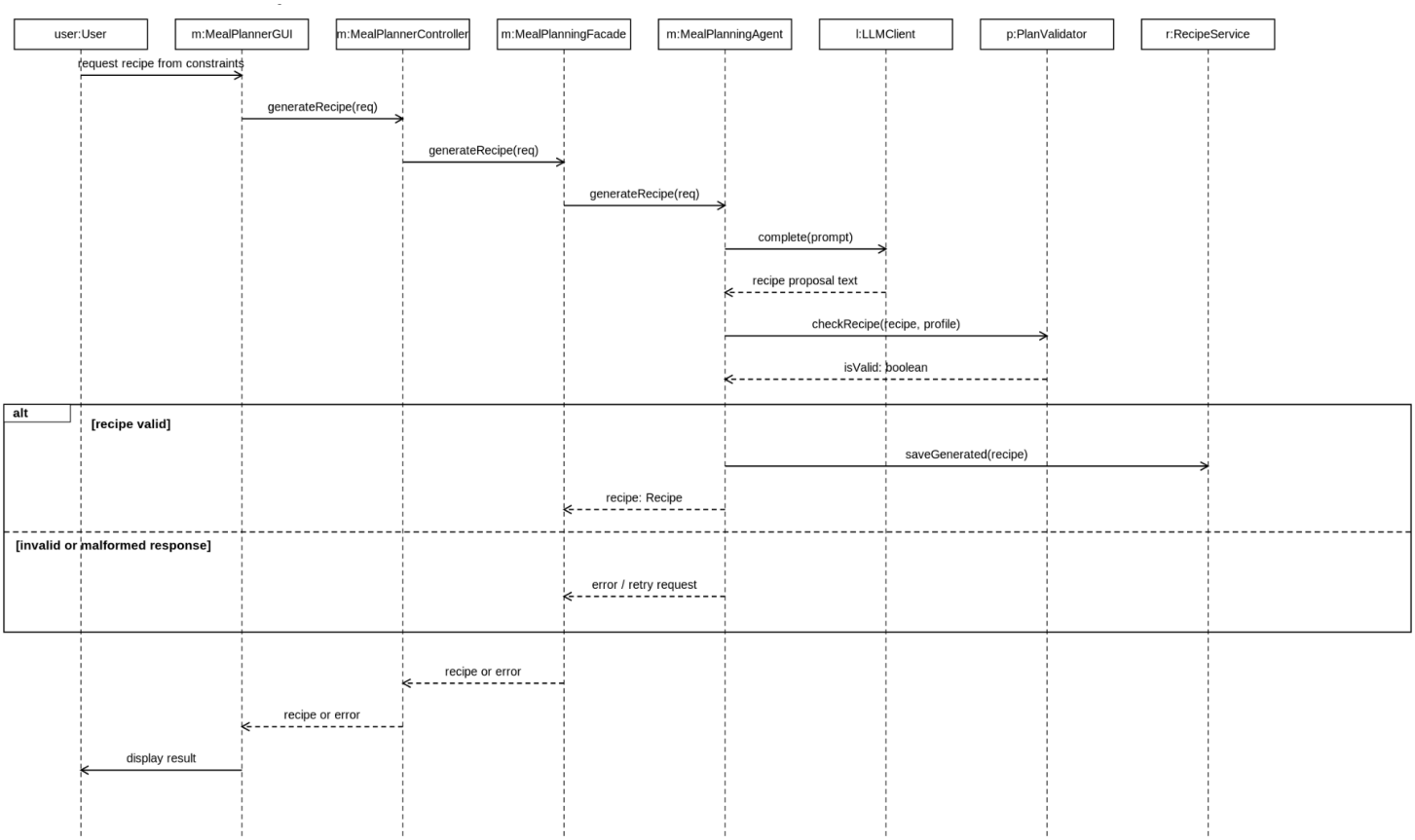


SD03 UC04, UC05, UC07–UC09 Recommendations, recipe generation, scaling, substitutions, leftovers (separate scenarios)

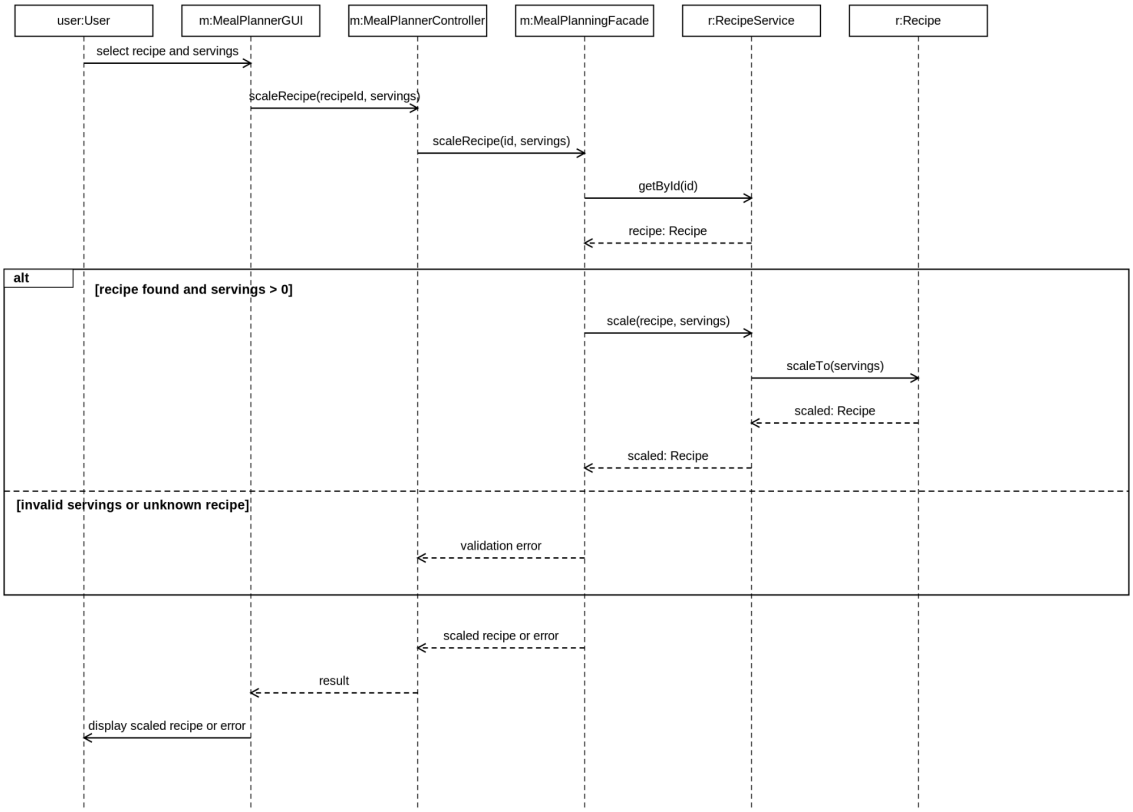
SD03-A UC04



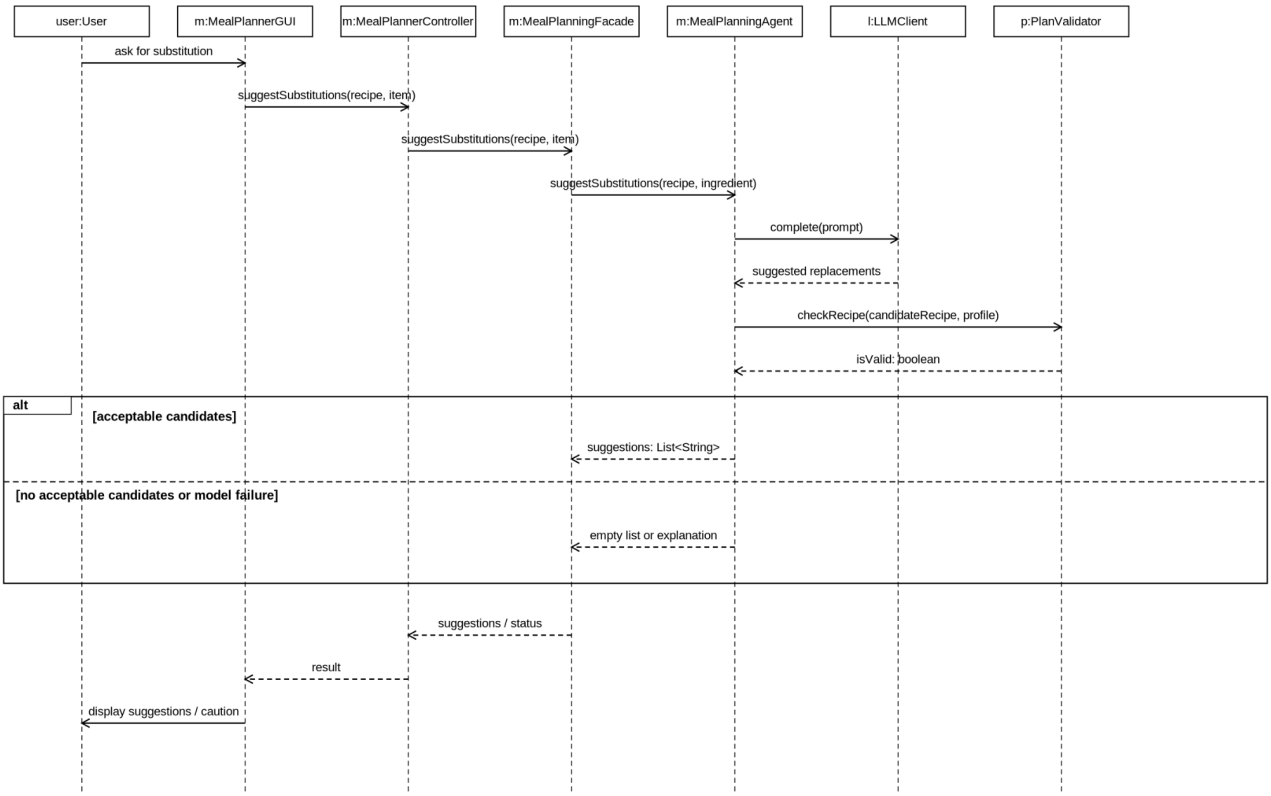
SD03-B UC05



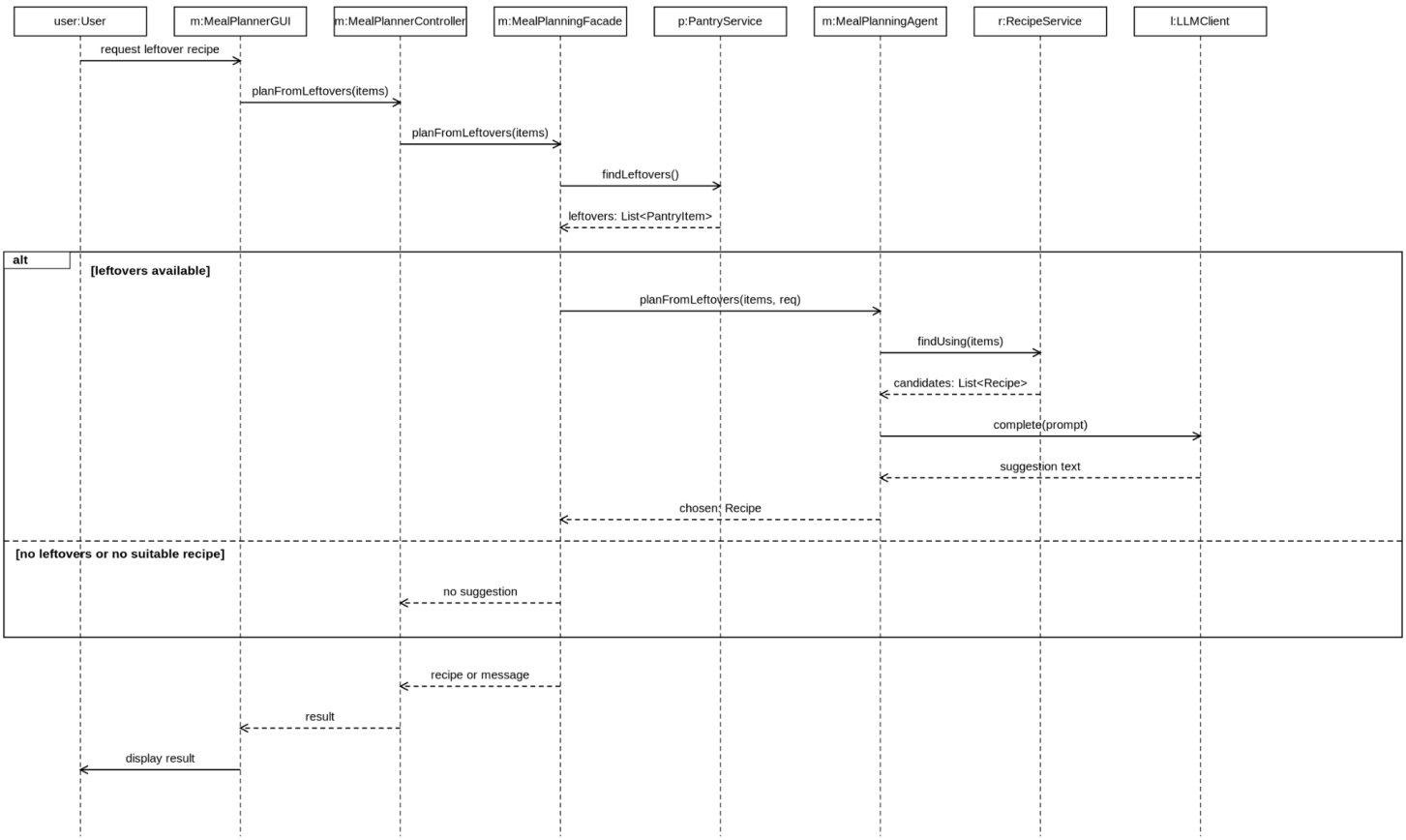
SD03-C UC07


user:User
m:MealPlannerGUI
m:MealPlannerController
m:MealPlanningFacade
r:RecipeService
r:Recipe
select recipe and servings
scaleRecipe(recipeId, servings)
scaleRecipe(id, servings)
getById(id)
recipe: Recipe
alt
[recipe found and servings > 0]
scale(recipe, servings)
scaleTo(servings)
scaled: Recipe
scaled: Recipe
[invalid servings or unknown recipe]
validation error
scaled recipe or error
result
display scaled recipe or error

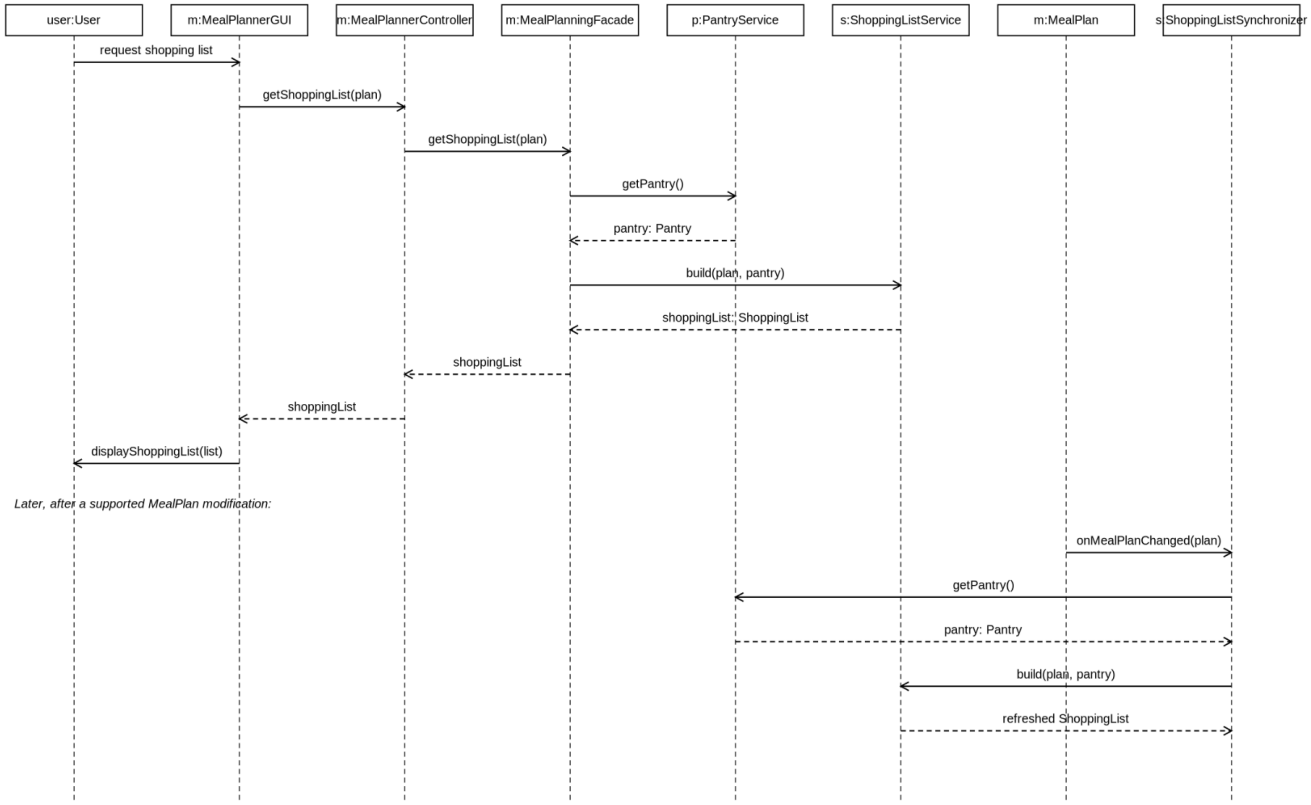
SD03- UC08


user:User
m:MealPlannerGUI
m:MealPlannerController
m:MealPlanningFacade
m:MealPlanningAgent
l:LLMClient
p:PlanValidator
ask for substitution
suggestSubstitutions(recipe, item)
suggestSubstitutions(recipe, item)
suggestSubstitutions(recipe, ingredient)
complete(prompt)
suggested replacements
checkRecipe(candidateRecipe, profile)
isValid: boolean
alt
[acceptable candidates]
suggestions: List<String>
[no acceptable candidates or model failure]
empty list or explanation
suggestions / status
result
display suggestions / caution

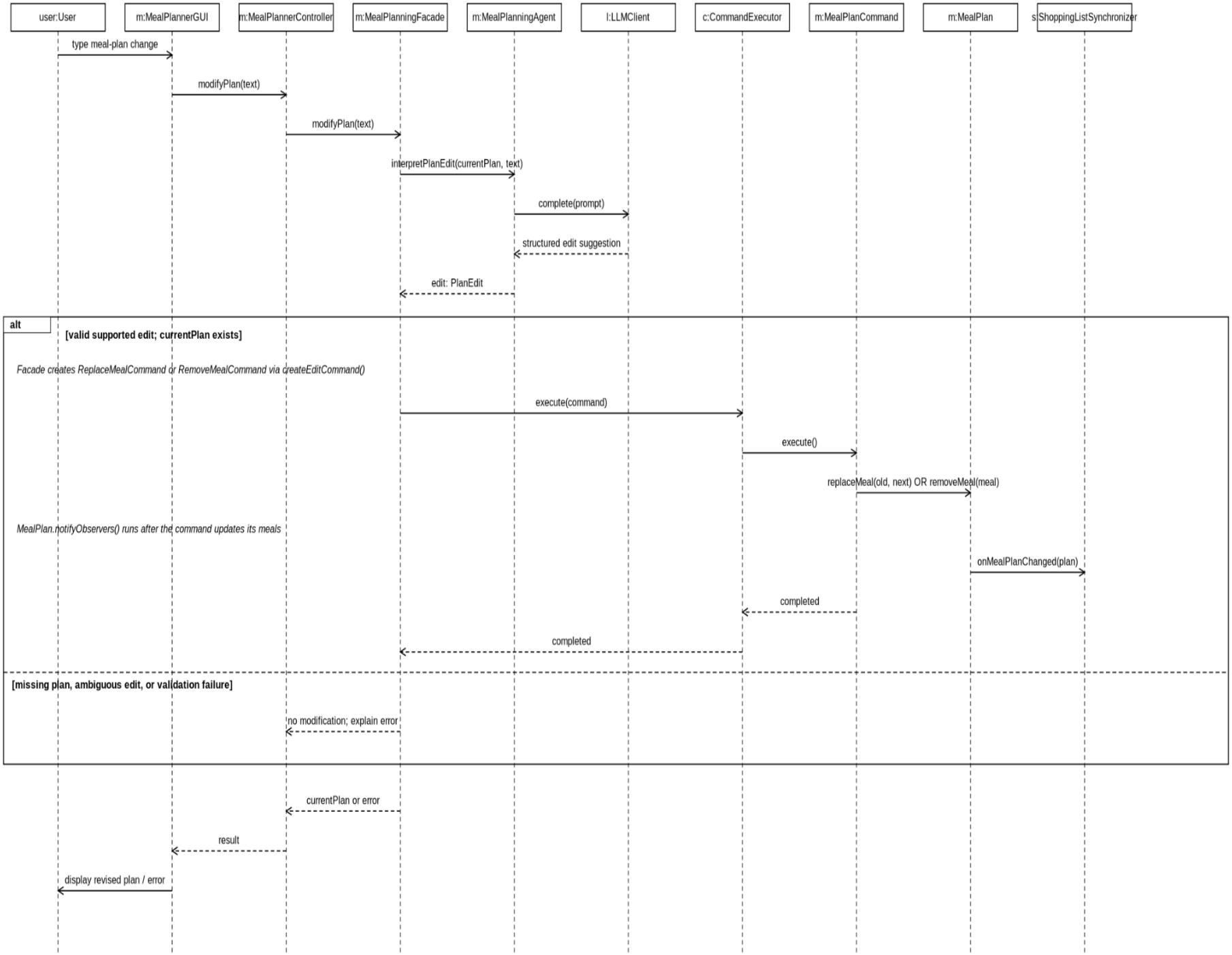
SD03-E UC09


user:User
m:MealPlannerGUI
m:MealPlannerController
m:MealPlanningFacade
p:PantryService
m:MealPlanningAgent
r:RecipeService
l:LLMClient
request leftover recipe
planFromLeftovers(items)
planFromLeftovers(items)
findLeftovers()
leftovers: List<PantryItem>
alt
[leftovers available]
planFromLeftovers(items, req)
findUsing(items)
candidates: List<Recipe>
complete(prompt)
suggestion text
chosen: Recipe
[no leftovers or no suitable recipe]
no suggestion
recipe or message
result
display result

SD04 UC10 Shopping-list creation and Observer-based synchronization



SD05 UC11 AI interpretation into PlanEdit; Command execution; shopping-list observer update


user:User
m:MealPlannerGUI
m:MealPlannerController
m:MealPlanningFacade
m:MealPlanningAgent
l:LLMClient
c:CommandExecutor
m:MealPlanCommand
m:MealPlan
s:ShoppingListSynchronizer
type meal-plan change
modifyPlan(text)
modifyPlan(text)
interpretPlanEdit(currentPlan, text)
complete(prompt)
structured edit suggestion
edit: PlanEdit
alt
[valid supported edit; currentPlan exists]
Facade creates ReplaceMealCommand or RemoveMealCommand via createEditCommand()
execute(command)
execute()
replaceMeal(old, next) OR removeMeal(meal)
MealPlan.notifyObservers() runs after the command updates its meals
onMealPlanChanged(plan)
completed
completed
[missing plan, ambiguous edit, or validation failure]
no modification; explain error
currentPlan or error
result
display revised plan / error

7. Design Pattern Explanations

MVC (Model–View–Controller)

Participating classes and roles	MealPlannerGUI and MealPlannerCLI are views; MealPlannerController is the controller; domain classes such as UserProfile, Recipe, Pantry, and MealPlan are the model.
Problem addressed / interaction	UI input and business operations would otherwise be tightly mixed. Both interfaces delegate to the same controller, which delegates to MealPlanningFacade instead of implementing meal-planning logic.
Why appropriate	Supports a GUI and CLI without duplicating business rules; UI changes do not require modifying the underlying domain objects.
Without this pattern	Without MVC, separate interfaces would likely duplicate validation and control flow.

Facade

Participating classes and roles	MealPlanningFacade provides unified operations to MealPlannerController and coordinates MealPlanningAgent, RecipeService, PantryService, ShoppingListService, and CommandExecutor.
Problem addressed / interaction	Recommendation, pantry, recipe search, list generation, and editing depend on different collaborators. The facade provides simple operations such as generatePlan(req), searchByIngredients(items), and modifyPlan(text).
Why appropriate	The controller remains independent of the detailed service orchestration.
Without this pattern	Without the facade, controllers and views would need to manage multiple service interactions and would become more tightly coupled.

Strategy

Participating classes and roles	MealPlanningStrategy defines rank(recipes, req, pantry); BudgetFirstStrategy and PantryFirstStrategy implement it; MealPlanningAgent holds a strategy reference and can call setStrategy(s).
Problem addressed / interaction	Weekly planning needs alternative recipe-ranking priorities without hard-coding a single ranking algorithm in the agent.
Why appropriate	Lets planning priorities vary while keeping the agent's generation workflow stable.
Without this pattern	Without Strategy, budget-first and pantry-first conditions would be scattered through the agent, making extension and testing harder.

Command

Participating classes and roles	MealPlanCommand defines execute() and undo(); ReplaceMealCommand and RemoveMealCommand implement it; CommandExecutor executes commands and stores a history; MealPlan is the receiver.
Problem addressed / interaction	AI-interpreted natural-language modifications must be executed consistently and allow undo. The agent returns a PlanEdit; MealPlanningFacade.createEditCommand(edit, plan) chooses the concrete command.
Why appropriate	Separates interpreting an edit from mutating the plan; makes supported changes undoable.
Without this pattern	Without Command, direct AI-driven mutations would complicate rollback, history, and consistent edit execution.

Observer

Participating classes and roles	MealPlan acts as the subject using attach(), detach(), and notifyObservers(); MealPlanObserver specifies onMealPlanChanged(plan); ShoppingListSynchronizer implements the observer and uses ShoppingListService and PantryService.
Problem addressed / interaction	Shopping lists must be refreshed when scheduled meals are changed by a Command, without embedding shopping-list calculations in MealPlan.
Why appropriate	Supports automatic recalculation while keeping the meal-plan domain object independent of the shopping-list algorithm.

Without this pattern	Without Observer, each meal-plan editing operation would need to manually update dependent components, increasing coupling.

8. Feature-to-Design Traceability

ID	Feature / Use case	Sequence	Patterns
F01	User preferences and constraints (UC01)	SD02-A	MVC, Facade
F02	Pantry and expiry management (UC02)	SD02-B	MVC, Facade
F03	Ingredient-based recipe search (UC03)	SD02-C	MVC, Facade
F04	Context-aware recommendations (UC04)	SD03-A; SD01 context	MVC, Facade, Strategy
F05	AI recipe generation (UC05)	SD03-B	Facade
F06	Weekly meal-plan generation (UC06)	SD01	MVC, Facade, Strategy, Observer
F07	Serving-size adjustment (UC07)	SD03-C	MVC, Facade
F08	Ingredient substitutions (UC08)	SD03-D	Facade
F09	Leftover-based planning (UC09)	SD03-E	Facade
F10	Automatic shopping list (UC10)	SD04	Facade, Observer
F11	Natural-language editing (UC11)	SD05	MVC, Facade, Command, Observer

Classes and Key Methods by Feature

F01 User preferences and constraints

Classes: MealPlannerController, MealPlanningFacade, UserProfile

Key methods: saveProfile(p); updatePreferences(values); setRestrictions(values)

F02 Pantry and expiry management

Classes: MealPlannerController, MealPlanningFacade, PantryService, Pantry, PantryItem

Key methods: managePantry(item); addOrUpdate(item); expiringWithin(days); findLeftovers()

F03 Ingredient-based recipe search

Classes: MealPlannerController, MealPlanningFacade, RecipeService, Recipe, PantryItem

Key methods: searchByIngredients(items); RecipeService.findUsing(items)

F04 Context-aware recommendations

Classes: MealPlannerController, MealPlanningFacade, MealPlanningAgent, RecipeService, MealPlanningStrategy

Key methods: recommendRecipes(req); recommend(req,pantry); search(query); rank(recipes,req,pantry)

F05 AI recipe generation

Classes: MealPlanningAgent, LLMClient, PlanValidator, RecipeService, Recipe

Key methods: generateRecipe(req); LLMClient.complete(prompt); checkRecipe(recipe,profile); saveGenerated(recipe)

F06 Weekly meal-plan generation

Classes: MealPlanningFacade, MealPlanningAgent, MealPlanningStrategy, LLMClient, PlanValidator, MealPlan

Key methods: generatePlan(req); generate(req,pantry); rank(recipes,req,pantry); validate(plan,req)

F07 Serving-size adjustment

Classes: MealPlannerController, MealPlanningFacade, RecipeService, Recipe

Key methods: scaleRecipe(recipeId,servings); getById(id); scale(recipe,servings); scaleTo(servings)

F08 Ingredient substitutions

Classes: MealPlanningAgent, LLMClient, Recipe, MealPlanningFacade

Key methods: suggestSubstitutions(recipe,ingredient); LLMClient.complete(prompt)

F09 Leftover-based planning

Classes: PantryService, PantryItem, MealPlanningAgent, Recipe

Key methods: findLeftovers(); planFromLeftovers(items)

F10 Automatic shopping list

Classes: ShoppingListService, MealPlan, Pantry, ShoppingList, MealPlanObserver, ShoppingListSynchronizer

Key methods: getShoppingList(plan); build(plan,pantry); missingFor(recipe,pantry); consolidate(items); onMealPlanChanged(plan)

F11 Natural-language editing

Classes: MealPlanningFacade, MealPlanningAgent, PlanEdit, EditOperation, CommandExecutor, ReplaceMealCommand, RemoveMealCommand, MealPlan, ShoppingListSynchronizer

Key methods: modifyPlan(text); interpretPlanEdit(plan,text); createEditCommand(edit,plan); execute(command); undoLast(); notifyObservers()

9. Feature Implementation Explanations

F01: Both the GUI and CLI send profile updates through MealPlannerController and MealPlanningFacade. The UserProfile class stores accepted preferences and dietary restrictions, which are then used by the recommendation and meal planning features. Keeping this information in one domain object ensures that different features can access consistent user preferences without maintaining separate copies. The design assumes that profile information is maintaied during the application session.

F02: PantryService manages a Pantry containing individual PantryItem objects. It handles inventory updates and provides methods to identify ingredients approaching expiry or marked as leftovers. These operations are deterministic because quantities, dates, and inventory records require consistent processing. Keeping pantry management in a dedicated service also allows the same inventory information to be reused by recipe search, meal planning, and shopping list generation.

F03: Ingredient based searches are passed from MealPlannerController through MealPlanningFacade to RecipeService.findUsing(). Keyword searches use RecipeService.search() instead. Both operations retrieve matching Recipe objects from the existing recipe collection rather than relying on the language model. I chose this approach to keep search results based on known recipe data and distinguish recipe retrieval from AI recipe generation.

F04: MealPlanningAgent receives the user's PlanRequest and Pantry context, retrieves candidate recipes through RecipeService, and ranks them using the selected MealPlanningStrategy. Unlike keyword searches, recommendations consider user preferences, restrictions, and planning priorities. The Strategy pattern allows ranking behavior to change between budget first and pantry first priorities without rewriting the recommendation process.

F05: MealPlanningAgent requests recipe content from the external language model through LLMClient. The generated response is converted into a structured Recipe and checked against available user restrictions using PlanValidator. An accepted recipe may then be saved through RecipeService.saveGenerated(). I separated recipe generation from validation so the application doesnt automatically treat every model response as suitable. Invalid or malformed results can be rejected or revised before being presented.

F06: MealPlanningFacade obtains the required pantry and planning context before invoking MealPlanningAgent.generate(). The selected BudgetFirstStrategy or PantryFirstStrategy ranks candidate recipes according to the user's planning priority, while the language model helps propose a weekly MealPlan. PlanValidator checks supported constraints, and the agent may request a revision if the proposed plan is invalid. Once a valid plan is accepted, the Facade stores it in currentPlan. This separates recipe ranking, AI assisted generation, and validation into distinct responsibilities, making the planning process easier to maintain and test.

F07: RecipeService retrieves the selected Recipe and scales its ingredient quantities using the original serving count. The adjusted recipe is returned with quantities calculated proportionally. I chose deterministic scaling because numerical calculations should produce consistent results rather than depend on AI generated estimates.

F08: MealPlanningAgent uses LLMClient to suggest alternatives for a selected recipe ingredient. The suggestions are checked against known dietary restrictions where sufficient information is available. These alternatives remain informational rather than automatically modifying the recipe. This approach gives users control over substitutions and avoids presenting suggestions as guaranteed safe when complete dietary or allergen information is unavailable.

F09: PantryService.findLeftovers() identifies PantryItem objects marked as leftovers. MealPlanningFacade passes these items and the relevant PlanRequest to MealPlanningAgent.planFromLeftovers(items, req), which proposes a suitable Recipe. Available expiry information is considered so that obviously expired items are not recommended. Reusing existing pantry records avoids maintaining a separate inventory for leftovers and allows the agent to consider the user's preferences and restrictions when making suggestions.

F10: ShoppingListService compares ingredient requirements from the planned recipes with quantities available in the Pantry. It identifies missing ingredients and consolidates repeated ShoppingListItem entries into a ShoppingList. When the MealPlan changes, it notifies the registered ShoppingListSynchronizer, which recalculates the list. I used the Observer pattern to keep the shopping list synchronized without requiring MealPlan to handle shopping list calculations directly, reducing coupling between the components.

F11: MealPlanningFacade retrieves the active currentPlan, and MealPlanningAgent interprets the user's natural language instruction into a structured PlanEdit. The Facade verifies the requested operation and creates either a ReplaceMealCommand or RemoveMealCommand. CommandExecutor executes the command and maintains a history that supports undo operations. After a successful modification, MealPlan notifies ShoppingListSynchronizer to update the shopping list. This design separates AI interpretation from actual data modification. The Command pattern provides controlled nd reversible changes. the Observer pattern maintains consistency between the meal plan and its shopping list.